Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по лабораторной работе №5 «Объектно-ориентированное программирование в Python»

по дисциплине «Программирование»

Вариант 2-8

Факультет: Безопасность информационных технологий

Группа: N3149

Выполнила: Беленькая Полина Александровна

Подпись:

Преподаватель: Гирик Алексей Валерьевич

Санкт-Петербург 2025

Задание

Разработать на языке Python для ОС Linux модуль, содержащий определение заданного типа, и тестовый модуль для pytest, содержащий набор тестов для проверки корректности определения типа.

Базовый класс: set. Формат данных в строке: дата и время(В формате HH:MM:SS dd.mm.yyyy).

Примеры работы программ на различных исходных данных:

```
>>> from prg5pabN3149 import DateTimeSet
>>> d = DateTimeSet()
>>> d.add("12:00:00 01.01.2025")
>>> print(d)
DateTimeSet({'12:00:00 01.01.2025'})
```

```
>>> d.add(123)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 28, in add
    self._validate(item)
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 18, in _validate
    raise TypeError("Допустимы только строки.")
TypeError: Допустимы только строки.
```

```
>>> d.add("2025-01-01 12:00:00")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 28, in add
    self._validate(item)
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 21, in _validate
    raise FormatError("Ожидаемый формат: HH:MM:SS dd.mm.yyyy.")
prg5pabN3149.FormatError: Ожидаемый формат: HH:MM:SS dd.mm.yyyy.
```

```
>>> d.add("12:00:00 31.02.2025")
Traceback (most recent call last):
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 23, in _validate
    datetime.strptime(item, "%H:%M:%S %d.%m.%Y")
  File "/usr/lib/python3.12/_strptime.py", line 554, in _strptime_datetime
    tt, fraction, gmtoff_fraction = _strptime(data_string, format)
                                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/_strptime.py", line 520, in _strptime
    julian = datetime_date(year, month, day).toordinal() - \
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: day is out of range for month

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 28, in add
    self._validate(item)
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 25, in _validate
    raise FormatError("Несуществующая дата.")
prg5pabN3149.FormatError: Несуществующая дата.
```

```
>>> d.add("13:00:00 02.02.2025")
>>> print(d)
DateTimeSet({'13:00:00 02.02.2025', '12:00:00 01.01.2025'})
>>> d.undo()
(d)>>> print(d)
DateTimeSet({'12:00:00 01.01.2025'})
```

```
>>> d.redo()
>>> print(d)
DateTimeSet({'13:00:00 02.02.2025', '12:00:00 01.01.2025'})
```

```
>>> d = DateTimeSet()
>>> d.undo()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/polina/prg5pabN3149/prg5pabN3149.py", line 91, in undo
    raise UndoError("Отсутствуют действия для отмены.")
prg5pabN3149.UndoError: Отсутствуют действия для отмены.
```

```
>>> print(d)
DateTimeSet()
>>> d.add("14:00:00 03.03.2025")
>>> d.remove("14:00:00 03.03.2025")
>>> d.undo()
>>> print(d)
DateTimeSet({'14:00:00 03.03.2025'})
```

Исходный код программы:

```
import re
from datetime import datetime


class FormatError(Exception): pass


class UndoError(Exception): pass


class RedoError(Exception): pass


class DateTimeSet(set):
    def __init__(self, iterable=None):
        super().__init__()
        self._history = []
        self._future = []
        if iterable:
            for item in iterable:
                self.add(item)

    def _validate(self, item):
        if not isinstance(item, str):
            raise TypeError("Допустимы только строки.")
        pattern = r'^(?:[01]\d|2[0-3]):[0-5]\d:[0-5]\d \d{2}\.\d{2}\.\d{4}$'
        if not re.match(pattern, item):
            raise FormatError("Ожидаемый формат: HH:MM:SS dd.mm.yyyy.")
        try:
            datetime.strptime(item, "%H:%M:%S %d.%m.%Y")
        except ValueError:
            raise FormatError("Несуществующая дата.")

    def add(self, item):
        self._validate(item)
        self._save_state()
        super().add(item)
        self._future.clear()

    def remove(self, item):
        self._save_state()
        super().remove(item)
        self._future.clear()

    def discard(self, item):
        self._save_state()
        super().discard(item)
        self._future.clear()

    def update(self, *others):
        items = set()
```

```
        for iterable in others:
            for item in iterable:
                self._validate(item)
                items.add(item)
        self._save_state()
        super().update(items)
        self._future.clear()

    def pop(self):
        self._save_state()
        item = super().pop()
        self._future.clear()
        return item

    def __ior__(self, other):
        self.update(other)
        return self

    def __iand__(self, other):
        self._save_state()
        result = super().__iand__(other)
        self._future.clear()
        return result

    def __isub__(self, other):
        self._save_state()
        result = super().__isub__(other)
        self._future.clear()
        return result

    def __ixor__(self, other):
        self._save_state()
        result = super().__ixor__(other)
        self._future.clear()
        return result

    def clear(self):
        self._save_state()
        super().clear()
        self._future.clear()

    def _save_state(self):
        self._history.append(set(self))

    def undo(self):
        if not self._history:
            raise UndoError("Отсутствуют действия для отмены.")
        self._future.append(set(self))
        previous_state = self._history.pop()
        super().clear()
        super().update(previous_state)

    def redo(self):
        if not self._future:
            raise RedoError("Отсутствуют действия для повтора.")
        self._history.append(set(self))
        next_state = self._future.pop()
        super().clear()
        super().update(next_state)
```

Тестирование:

```
import pytest
from prg5pabN3149 import FormatError, UndoError, RedoError, DateTimeSet


def test_add_valid():
    dts = DateTimeSet()
```

```python
    dts.add('12:24:01 01.02.2001')
    assert '12:24:01 01.02.2001' in dts


def test_add_invalid_type():
    dts = DateTimeSet()
    with pytest.raises(TypeError):
        dts.add(123)


def test_add_invalid_format():
    dts = DateTimeSet()
    with pytest.raises(FormatError):
        dts.add('2025-05-12 14:30')


def test_undo_redo():
    dts = DateTimeSet()
    dts.add('12:24:01 01.02.2001')
    dts.undo()
    assert '12:24:01 01.02.2001' not in dts
    dts.redo()
    assert '12:24:01 01.02.2001' in dts


def test_undo_error():
    dts = DateTimeSet()
    with pytest.raises(UndoError):
        dts.undo()


def test_redo_error():
    dts = DateTimeSet()
    with pytest.raises(RedoError):
        dts.redo()


def test_remove_and_undo():
    dts = DateTimeSet()
    dts.add('12:00:00 01.01.2020')
    dts.remove('12:00:00 01.01.2020')
    assert '12:00:00 01.01.2020' not in dts
    dts.undo()
    assert '12:00:00 01.01.2020' in dts


def test_discard_and_undo():
    dts = DateTimeSet()
    dts.add('13:00:00 01.01.2020')
    dts.discard('13:00:00 01.01.2020')
    dts.undo()
    assert '13:00:00 01.01.2020' in dts


def test_clear_and_undo():
    dts = DateTimeSet()
    dts.add('14:00:00 01.01.2020')
    dts.clear()
    assert len(dts) == 0
    dts.undo()
    assert '14:00:00 01.01.2020' in dts


def test_len_and_contains():
    dts = DateTimeSet()
    dts.add('15:00:00 01.01.2020')
    assert len(dts) == 1
```

```
    assert '15:00:00 01.01.2020' in dts


def test_iteration():
    dts = DateTimeSet()
    values = ['10:00:00 01.01.2020', '11:00:00 01.01.2020']
    for val in values:
        dts.add(val)
    assert set(dts) == set(values)
```